# LAPORAN UJIAN TENGAH SEMESTER

# KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

# SISTEM MANAJEMEN ASET LABORATORIUM DENGAN CI/CD DAN CONTAINERIZATION




**Disusun oleh:**

| | |
|---|---|
| **Nama** | : Hafidz Rizqullah Prasetya |
| **NIM** | : 24/535493/SV/24243 |
| **Kelas** | : PL5A1 |
| **Dosen Pengampu** | : Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs. |
| | Galih Malela Damaraji, S.Pd., M.Eng. |

**PROGRAM STUDI D-IV**

**TEKNOLOGI REKAYASA PERANGKAT LUNAK**

**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**

**SEKOLAH VOKASI**

**UNIVERSITAS GADJAH MADA**

**YOGYAKARTA**

**2026**

# Daftar Isi

**Daftar Isi** . . . . . **2**

**Tujuan Ujian** . . . . . **3**

**Dasar Teori** . . . . . **4**
2.1 Arsitektur RESTful API dan Otentikasi Token . . . . . 4
2.2 Aplikasi Halaman Tunggal dan Klien HTTP Reaktif . . . . . 4
2.3 Manajemen Repositori Git dan Proteksi Cabang . . . . . 5
2.4 Otomatisasi Alur Kerja dengan GitHub Actions . . . . . 5
2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage . . . . . 5

**Hasil dan Pembahasan** . . . . . **6**
3.1 Backend Laravel RESTful API (CPMK 1) . . . . . 6
3.1.1 Perancangan Skema Data dan Model . . . . . 6
3.1.2 Otentikasi Token Laravel Sanctum . . . . . 6
3.1.3 Implementasi Endpoint dan Status Code HTTP . . . . . 7
3.1.4 Koleksi Postman dan Pengujian Otomatis . . . . . 7
3.2 Frontend Vue.js dan Axios (CPMK 1) . . . . . 9
3.2.1 Konfigurasi Variabel Lingkungan . . . . . 9
3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token . . . . . 10
3.2.3 Dasbor Metrik dan Operasi CRUD Aset . . . . . 10
3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2) . . . . . 11
3.3.1 Struktur Repositori dan Strategi Percabangan . . . . . 12
3.3.2 Konfigurasi Branch Protection Rules . . . . . 14
3.3.3 Pipeline Continuous Integration (CI) . . . . . 15
3.4 Continuous Deployment ke GHCR (CPMK 3) . . . . . 16
3.4.1 Pembatasan Eksekusi Deploy pada Cabang Utama . . . . . 16
3.4.2 Manajemen Rahasia dengan GitHub Secrets . . . . . 16
3.4.3 Tag Citra Berdasarkan Commit SHA dan Registri GHCR . . . . . 17
3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5) . 19
3.5.1 Dockerfile Backend dengan Optimasi Layer Caching . . . . . 20
3.5.2 Dockerfile Frontend Multi-Stage Build . . . . . 21

**Kesimpulan** . . . . . **22**

**Daftar Pustaka** . . . . . **23**

# Tujuan Ujian

Sesuai arahan pada lembar soal Ujian Tengah Semester mata kuliah *Konstruksi dan Evolusi Perangkat Lunak* Semester Gasal Tahun Akademik 2026/2027, tujuan pengerjaan ujian ini adalah:

1. Mahasiswa mampu merancang dan mengimplementasikan layanan backend RESTful API berbasis Laravel dengan tema mandiri yang dilengkapi otentikasi token Laravel Sanctum, operasi CRUD lengkap, dan kesesuaian kode status HTTP 201, 422, serta 401 (CPMK 1).

2. Mahasiswa mampu membangun antarmuka web interaktif berbasis Vue.js 3 dan Axios yang terhubung ke backend, menyimpan token otentikasi pada penyimpanan lokal, mengelola data aset, dan membaca alamat API melalui variabel lingkungan (CPMK 1).

3. Mahasiswa mampu mengelola repositori publik Git di GitHub dengan struktur `backend/` dan `frontend/`, menerapkan aturan proteksi cabang `main` dan `dev`, serta menyusun alur pengujian otomatis menggunakan GitHub Actions (CPMK 2).

4. Mahasiswa mampu menerapkan alur penerapan berkelanjutan (*Continuous Deployment*) pada GitHub Actions yang dibatasi hanya untuk cabang `main`, mengamankan konfigurasi sensitif dengan GitHub Secrets, serta mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`) menggunakan tag commit SHA (CPMK 3).

5. Mahasiswa mampu menyusun berkas *Dockerfile* backend dengan optimasi *layer caching* dependensi Composer, serta berkas *Dockerfile* frontend dengan teknik *multi-stage build* berbasis Nginx yang bebas dari direktori dependensi kerja (CPMK 4 dan CPMK 5).

# Dasar Teori

## 2.1 Arsitektur RESTful API dan Otentikasi Token

Arsitektur REST (*Representational State Transfer*) mengandalkan protokol HTTP standar untuk pertukaran representasi sumber daya secara terstruktur. Layanan REST bersifat *stateless*, yang berarti setiap permintaan dari klien harus memuat seluruh konteks otentikasi yang diperlukan tanpa bergantung pada sesi yang disimpan di memori peladen [1].

Laravel Sanctum menyediakan mekanisme penerbitan token akses pribadi (*Personal Access Token*) yang efisien untuk aplikasi halaman tunggal (SPA) maupun klien mandiri. Ketika pengguna berhasil melakukan verifikasi kredensial, peladen menghasilkan string acak berbobot kriptografis dan menyimpannya dalam bentuk hash di basis data. Klien menyimpan nilai teks biasa dari token tersebut dan menyertakannya pada tajuk HTTP `Authorization: Bearer <token>` pada setiap permintaan berikutnya. Penerapan kode status HTTP baku menjadi penanda kepatuhan arsitektur, seperti 201 saat data berhasil dibuat, 422 saat validasi masukan tidak terpenuhi, dan 401 saat permintaan tidak menyertakan identitas otentikasi yang sah [1].

## 2.2 Aplikasi Halaman Tunggal dan Klien HTTP Reaktif

Kerangka kerja Vue.js 3 mengusung paradigma reaktif berbasis komponen yang memungkinkan pembaruan antarmuka pengguna secara modular tanpa perlu memuat ulang seluruh halaman peramban [2].

Integrasi antara komponen visual dan peladen REST dijembatani oleh pustaka Axios. Melalui mekanisme pencegat permintaan (*request interceptor*), token yang tersimpan di `localStorage` dapat disisipkan secara konsisten pada tajuk setiap panggilan jaringan. Sebaliknya, pencegat respons (*response interceptor*) bertugas mendeteksi kode status 401 guna membersihkan data sesi lokal ketika otentikasi kedaluwarsa. Standar rekayasa perangkat lunak mengharuskan alamat URL peladen dipisahkan dari logika komponen dan dibaca melalui berkas konfigurasi lingkungan (*environment variable*) [3].

## 2.3 Manajemen Repositori Git dan Proteksi Cabang

Dalam metodologi pengembangan perangkat lunak modern, cabang `main` berfungsi sebagai representasi kode produksi yang stabil, sedangkan cabang `dev` digunakan untuk konsolidasi fitur yang sedang dikembangkan. Pekerjaan penambahan modul baru diisolasi pada cabang fitur dan diintegrasikan melalui *Pull Request* (PR) [4].

Fitur *Branch Protection Rules* di GitHub memungkinkan penegakan kebijakan integrasi dengan melarang pengunggahan langsung (*direct push*). Kebijakan ini mewajibkan seluruh pengujian terotomatisasi (*status checks*) berstatus lulus sebelum perubahan diizinkan untuk digabungkan, sehingga mencegah masuknya kesalahan regresi ke cabang utama [4].

## 2.4 Otomatisasi Alur Kerja dengan GitHub Actions

GitHub Actions menyediakan infrastruktur otomatisasi siklus hidup perangkat lunak yang merespons berbagai kejadian pada repositori. Alur kerja didefinisikan dalam format deklaratif YAML pada direktori `.github/workflows/` [5].

Pemisahan tugas antara pengujian backend (*backend test*) dan kompilasi frontend (*frontend build*) memungkinkan eksekusi secara paralel guna mempercepat umpan balik. Pada tahap lanjutan, tahap penerapan (*deploy job*) dikonfigurasi secara bersyarat sehingga hanya dieksekusi saat kode berhasil digabungkan ke cabang `main`, sedangkan pada cabang lain alur kerja otomatis berhenti setelah pengujian selesai [5].

## 2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage

Teknologi kontainer Docker mengisolasi aplikasi beserta pustaka pendukungnya ke dalam satu kesatuan citra yang dapat dijalankan secara konsisten di berbagai lingkungan [6].

Penyusunan berkas *Dockerfile* yang efisien memanfaatkan prinsip pengurutan lapisan (*layer caching*). Dengan menyalin berkas dependensi seperti `composer.json` dan `package.json` sebelum menyalin kode sumber aplikasi, tahap pengunduhan pustaka tidak perlu diulang selama manifes dependensi tidak berubah. Pada frontend, teknik *multi-stage build* memisahkan tahap kompilasi berbasis Node.js dari tahap penyajian statis berbasis peladen Nginx yang berukuran ringkas, sehingga berkas sementara dan dependensi pembangun tidak ikut terbawa ke citra produksi [6, 7].

# Hasil dan Pembahasan

## 3.1 Backend Laravel RESTful API (CPMK 1)

Pengerjaan soal nomor 1 difokuskan pada perancangan dan implementasi layanan RESTful API menggunakan kerangka kerja Laravel dengan tema Sistem Manajemen Aset Laboratorium. Sistem ini menangani inventaris peralatan komputasi, perangkat jaringan, dan instrumen laboratorium di lingkungan Departemen Teknik Elektro dan Informatika Sekolah Vokasi UGM.

### 3.1.1 Perancangan Skema Data dan Model

Sumber daya utama pada backend adalah `Asset`. Tabel `assets` dirancang untuk menyimpan atribut nama perangkat, kode inventaris unik, kategori, status operasional, nilai perolehan, dan lokasi laboratorium. Bentuk migrasinya tercantum pada Listing 1.

```
Schema::create('assets', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('code')->unique();
    $table->string('category');
    $table->string('status')->default('tersedia');
    $table->decimal('price', 15, 2);
    $table->string('location')->nullable();
    $table->timestamps();
});
```

Listing 1: Migrasi skema tabel assets pada backend Laravel.

Model `Asset` dilengkapi atribut `$fillable` guna mencegah anomali *mass assignment*, serta *type casting* agar atribut harga terkonversi menjadi format desimal presisi pada respons JSON.

### 3.1.2 Otentikasi Token Laravel Sanctum

Modul otentikasi dikelola oleh `AuthController`. Pengguna masuk ke sistem melalui rute publik `POST /api/login`. Jika kredensial cocok, peladen menerbitkan token teks biasa

menggunakan metode `createToken()` dari trait `HasApiTokens`. Token tersebut dikembalikan ke klien bersama data identitas dasar pengguna.

Seluruh rute sumber daya dilindungi oleh middleware `auth:sanctum`. Apabila rute privat diakses tanpa menyertakan token atau dengan token yang tidak sah, penanganan pengecualian pada `bootstrap/app.php` secara otomatis mengembalikan respons JSON dengan status **401 Unauthorized**. Rute `POST /api/logout` mencabut token aktif dari basis data sehingga token tersebut tidak dapat digunakan lagi.

### 3.1.3 Implementasi Endpoint dan Status Code HTTP

Operasi data pada `AssetController` mengembalikan kode status HTTP berikut:

1. `200 OK`: Dikembalikan saat memuat daftar aset (`GET /api/assets`), detail satu aset (`GET /api/assets/{id}`), memperbarui data (`PUT /api/assets/{id}`), dan menghapus data (`DELETE /api/assets/{id}`).

2. `201 Created`: Dikembalikan saat pembuatan entitas aset baru berhasil disimpan ke basis data (`POST /api/assets`).

3. `422 Unprocessable Content`: Dikembalikan saat data yang dikirimkan tidak memenuhi aturan validasi (seperti nama kosong, kode duplikat, atau format harga negatif).

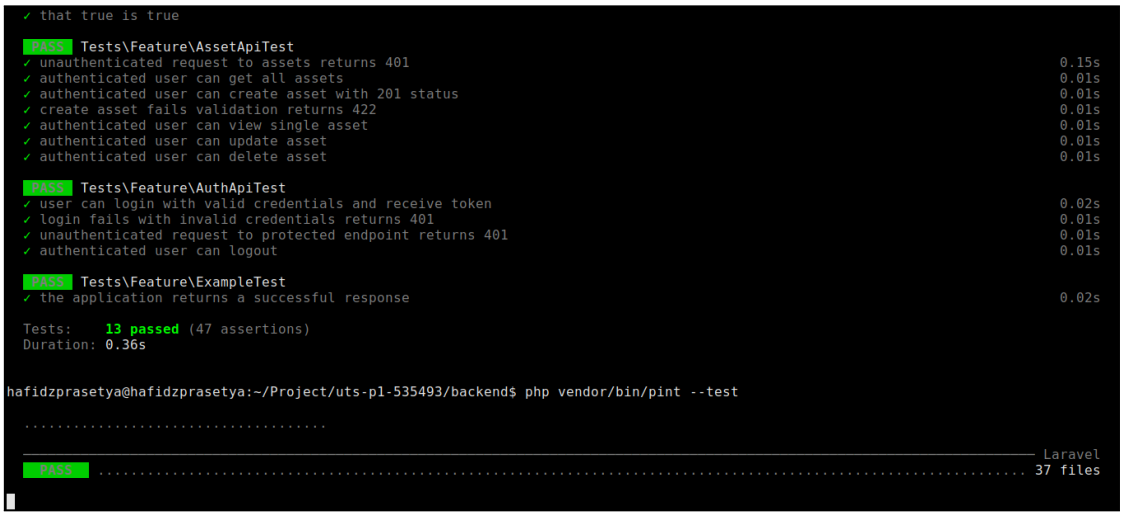
4. `401 Unauthorized`: Dikembalikan saat rute privat diakses tanpa Bearer Token yang sah.

### 3.1.4 Koleksi Postman dan Pengujian Otomatis

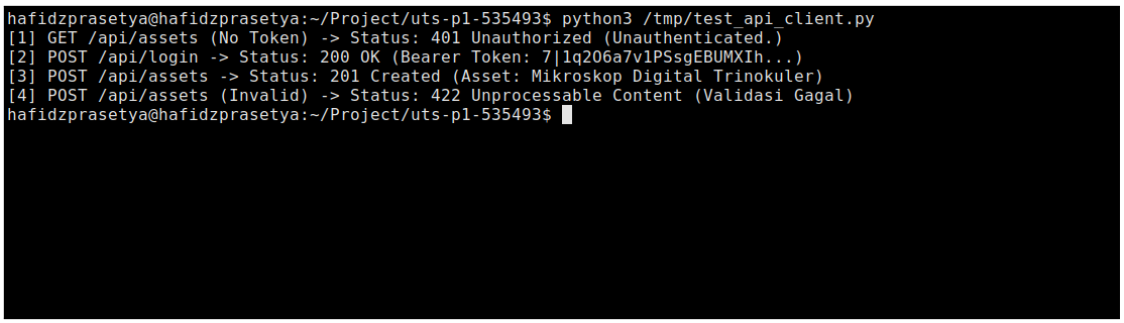
Seluruh rute API telah diuji menggunakan koleksi Postman yang diekspor ke dalam repositori pada berkas `backend/postman/uts_kepl_asset_api.postman_collection.json`. Koleksi ini mencakup skrip uji otomatis yang menyimpan token sesi ke variabel koleksi secara instan setelah panggilan login berhasil.

Keandalan backend diverifikasi secara terotomatisasi melalui berkas pengujian fitur (*Feature Tests*) pada `tests/Feature/AuthApiTest.php` dan `tests/Feature/AssetApiTest.php`. Ketiga belas pengujian dengan 47 asersi lulus seluruhnya, dan pemeriksaan gaya kode Laravel Pint juga bersih. Keluaran kedua perintah tersebut dapat dilihat pada Gambar 1.

Kepatuhan kode status HTTP diuji langsung melalui klien API pada terminal Linux; empat respons yang diharapkan muncul berurutan pada Gambar 2.




Gambar 1: Hasil eksekusi pengujian otomatis PHPUnit dan pemeriksaan gaya kode Laravel Pint pada terminal Linux.




Gambar 2: Verifikasi pemenuhan kode status HTTP 401, 200, 201, dan 422 melalui pemanggilan klien API di terminal Linux.

## 3.2 Frontend Vue.js dan Axios (CPMK 1)

Pengerjaan soal nomor 2 mencakup pembangunan antarmuka web modern menggunakan Vue.js 3, bundler Vite, dan pustaka HTTP Axios.

### 3.2.1 Konfigurasi Variabel Lingkungan

Untuk memenuhi ketentuan soal 2.c, alamat basis API tidak ditulis mati di dalam kode komponen. Alamat tersebut dikonfigurasi melalui berkas `.env` dan diakses melalui `import.meta.env.VITE_API_BASE_URL`, dengan konfigurasi seperti pada Listing 2.

```
import axios from 'axios';

const apiBaseUrl = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000/api';

const apiClient = axios.create({
  baseURL: apiBaseUrl,
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});

apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('auth_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

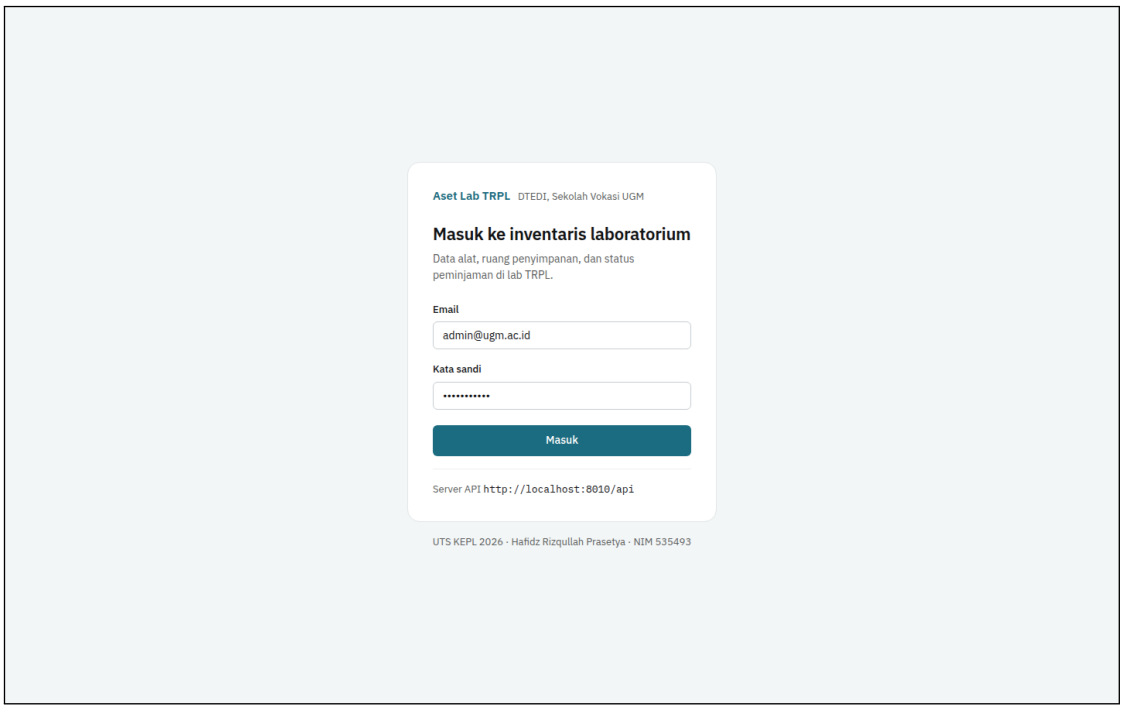
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('auth_token');
      localStorage.removeItem('auth_user');
      window.dispatchEvent(new CustomEvent('auth:unauthorized'));
    }
    return Promise.reject(error);
  }
);

export default apiClient;
export { apiBaseUrl };
```

Listing 2: Konfigurasi klien Axios dengan pembacaan variabel lingkungan.

### 3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token

Saat pengguna membuka aplikasi, sistem memeriksa ketersediaan sesi di penyimpanan lokal. Jika sesi belum tersedia, formulir login ditampilkan sebagaimana terlihat pada Gambar 3. Di bawah formulir tercetak alamat server API yang sedang dipakai. Nilai itu dibaca dari `VITE_API_BASE_URL`; saat tangkapan layar diambil, variabel tersebut diarahkan ke `http://localhost:8010/api` karena port 8000 di mesin pengembang sedang dipakai layanan lain.



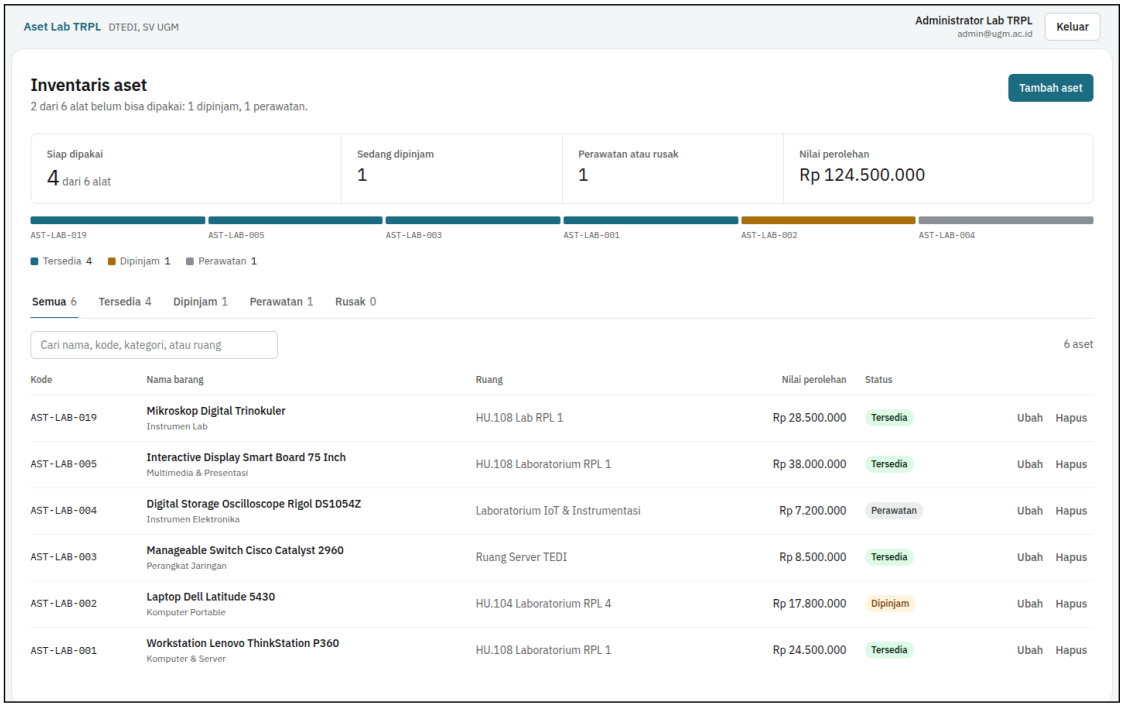

Gambar 3: Halaman login antarmuka web Vue.js dengan pembacaan dinamis URL API dari variabel lingkungan.

Ketika formulir dikirimkan, token yang diterima dari backend disimpan ke dalam `localStorage` dengan kunci `auth_token`. Pencegat Axios secara otomatis menyertakan token ini pada tajuk otentikasi di setiap permintaan berikutnya.

### 3.2.3 Dasbor Metrik dan Operasi CRUD Aset

Setelah login, pengguna masuk ke halaman inventaris. Baris judul langsung menyebut berapa alat yang belum bisa dipakai beserta alasannya, misalnya satu dipinjam dan satu dalam perawatan. Di bawahnya terdapat ringkasan jumlah alat siap pakai, dipinjam, dalam perawatan atau rusak, dan total nilai perolehan, lalu strip kesiapan yang menggambar satu batang untuk setiap kode aset menurut warna statusnya. Daftar aset dapat disaring lewat tab status

dan kolom pencarian (nama, kode, kategori, atau ruang). Nilai perolehan dicetak rata kanan dengan angka tabular agar mudah dibandingkan (Gambar 4).



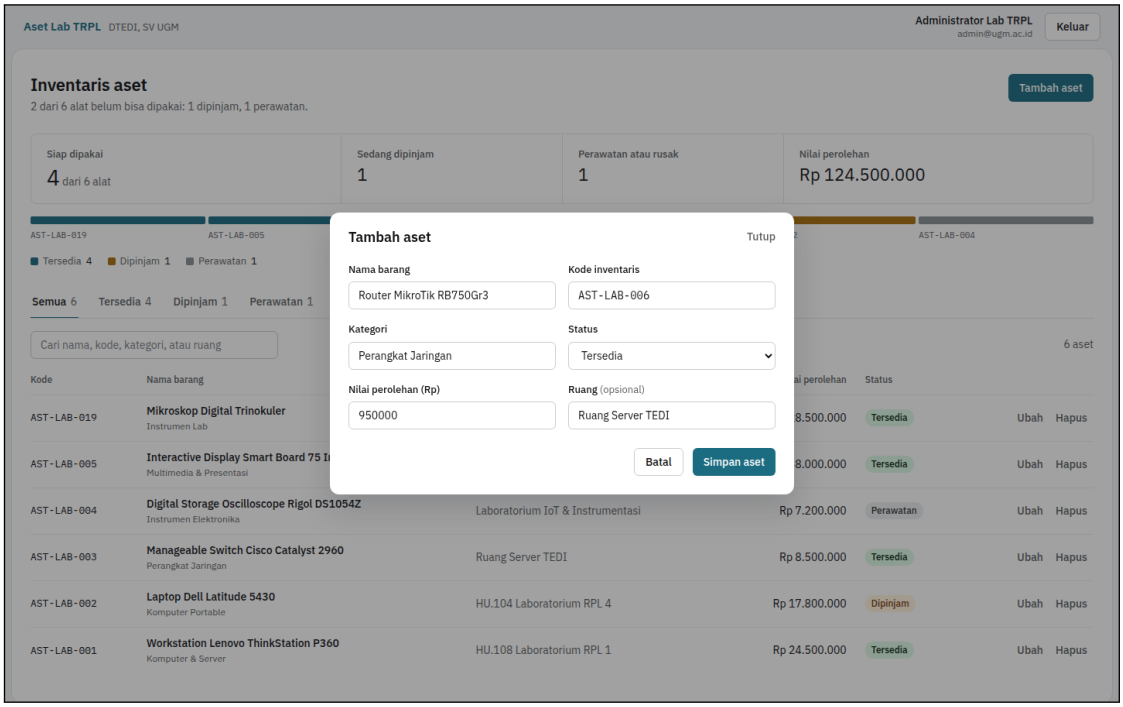

Gambar 4: Dasbor manajemen inventaris aset laboratorium pada antarmuka web Vue.js.

Penambahan aset dilakukan melalui modal yang mengirim data ke `POST /api/assets`. Ketika peladen merespons 201 Created, aset baru langsung muncul di tabel dan angka ringkasan ikut berubah tanpa memuat ulang halaman. Jika peladen membalas 422, pesan validasi dari Laravel ditampilkan di bawah kolom yang bermasalah. Modal dapat ditutup dengan tombol Batal, tombol Tutup, atau tombol Escape. Gambar 5 memperlihatkan modal tersebut ketika sedang diisi.

Pada layar ponsel, ringkasan berubah menjadi dua kolom, tab status dapat digeser, dan setiap baris tabel menjadi kartu dua baris berisi kode, status, nama, nilai, serta tombol Ubah dan Hapus berukuran sentuh 44 piksel (Gambar 6).

## 3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2)

Sesuai soal nomor 3, seluruh kode sumber dikelola pada repositori publik bernama `uts-p1-535493` di akun GitHub pribadi (`https://github.com/hafidzrizqullahprasetya/uts-p1-535493`). Repositori memuat folder `backend/` dan `frontend/`, serta menerapkan aturan percabangan yang terisolasi.




Gambar 5: Modal interaktif penambahan data aset laboratorium baru pada antarmuka frontend.

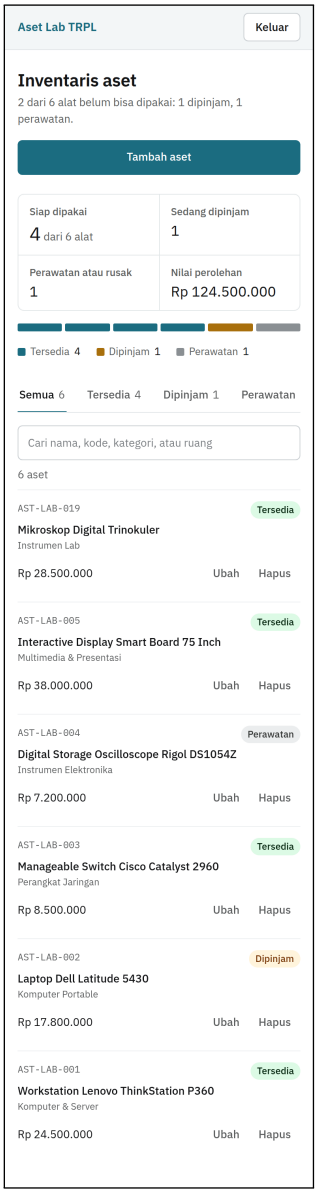
### 3.3.1 Struktur Repositori dan Strategi Percabangan

Repositori diatur dengan tiga tingkatan cabang:

1. `main`: Cabang produksi utama yang dilindungi secara ketat.
2. `dev`: Cabang integrasi pengembangan pra-produksi.
3. `feat/backend-and-frontend` dan `feat/ui-refresh-and-secrets`: Cabang fitur, masing-masing untuk satu paket perubahan.

Implementasi awal backend dan frontend masuk ke `dev` melalui Pull Request #1, lalu dari `dev` ke `main` melalui Pull Request #2. Perombakan antarmuka, pembaruan PHP ke 8.5.12, dan perbaikan penggunaan GitHub Secrets menempuh jalur yang sama melalui Pull Request #3 dan #4. Komit `f83fed0` (perapian komentar) sempat didorong langsung ke `main` karena saat itu aturan proteksi belum mewajibkan Pull Request dan admin masih bisa melewatinya. Celah ini kemudian ditutup dengan aturan pada subbab berikutnya. Listing 3 memuat graf riwayat komit cabang `main`.

```
$ git log --graph --oneline -n 9
*   eeab6d9 Merge pull request #4 from hafidzrizqullahprasetya/dev
|\
| *   3918efe Merge pull request #3 from .../feat/ui-refresh-and-secrets
| |\
| | * 6d49098 feat: rombak antarmuka inventaris dan perbarui php ke 8.5.12
| |/
|/|
* | f83fed0 style: bersihkan komentar kode dan rapikan berkas sumber
```

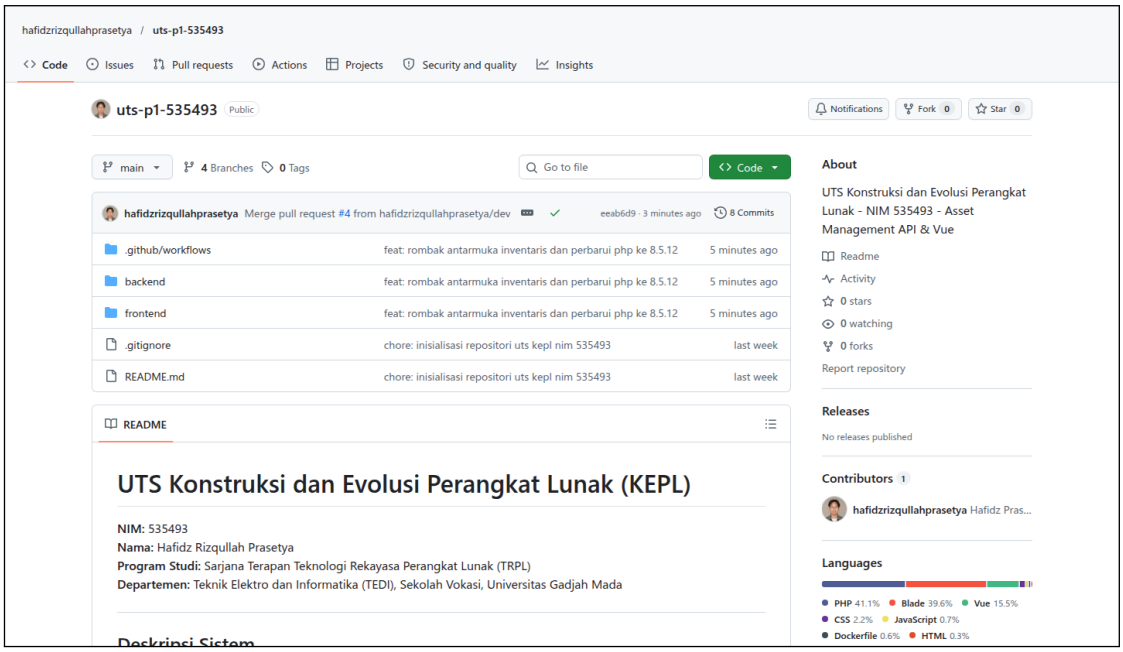



Gambar 6: Tampilan halaman inventaris pada lebar layar ponsel 390 piksel.

```
* | 085dd03 Merge pull request #2 from hafidzrizqullahprasetya/dev
|\|
| * 7d1ddd6 Merge pull request #1 from .../feat/backend-and-frontend
|/|
| * 50ce732 feat: implementasi backend laravel sanctum dan frontend vue
|/
* 07c7b52 chore: inisialisasi repositori uts kepl nim 535493
```

Listing 3: Graf riwayat komit cabang main setelah Pull Request #4.

Beranda repositori publik tersebut, termasuk struktur direktori `backend/` dan `frontend/`, tampak pada Gambar 7.



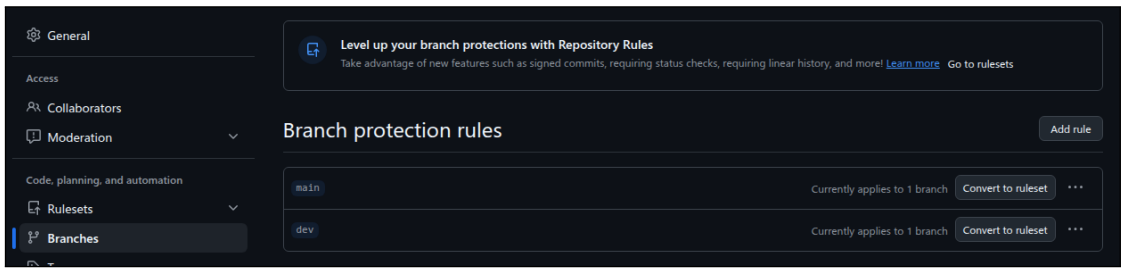

Gambar 7: Tampilan repositori publik `uts-p1-535493` pada platform GitHub.

### 3.3.2 Konfigurasi Branch Protection Rules

Cabang `main` dan `dev` diproteksi untuk menolak pengunggahan langsung (*direct push*). Aturan proteksi mewajibkan:

1. Integrasi hanya dapat dilakukan melalui mekanisme Pull Request.
2. Pemeriksaan status (*status checks*) harus lulus sebelum penggabungan, yaitu pekerjaan `Backend Test` dan `Frontend Build`.
3. Opsi sinkronisasi cabang mutakhir (*strict check*) aktif.
4. Aturan berlaku juga untuk administrator repositori (*enforce admins*), sehingga pemilik repositori pun tidak bisa melewatinya.

Gambar 8 menampilkan dua aturan proteksi yang aktif pada antarmuka GitHub.




Gambar 8: Pengaturan Branch Protection Rules untuk cabang `main` dan `dev` pada repositori GitHub.

Isi aturan tidak terlihat pada daftar di atas, sehingga konfigurasinya diperiksa lewat GitHub CLI. Keluarannya menunjukkan Pull Request wajib, pemeriksaan status bersifat ketat dan mencakup kedua pekerjaan CI, aturan berlaku bagi admin, serta pengunggahan paksa dan penghapusan cabang ditolak (Listing 4). Cabang `dev` memberikan keluaran yang sama.

```
$ gh api repos/hafidzrizqullahprasetya/uts-p1-535493/branches/main/protection \
    --jq '{strict: .required_status_checks.strict,
           checks: .required_status_checks.contexts,
           pr_required: (.required_pull_request_reviews != null),
           enforce_admins: .enforce_admins.enabled,
           force_push: .allow_force_pushes.enabled,
           deletion: .allow_deletions.enabled}'
{
  "strict": true,
  "checks": ["Backend Test", "Frontend Build"],
  "pr_required": true,
  "enforce_admins": true,
  "force_push": false,
  "deletion": false
}
```

Listing 4: Verifikasi konfigurasi proteksi cabang main melalui GitHub CLI.

### 3.3.3 Pipeline Continuous Integration (CI)

Alur kerja integrasi berkelanjutan didefinisikan pada berkas `.github/workflows/ci.yml`. Berkas ini memuat dua pekerjaan pengujian yang berjalan secara paralel:

1. **Backend Test:** Menyiapkan lingkungan PHP 8.5, memasang dependensi Composer dari cache, membaca `APP_KEY` dari GitHub Secrets, menginisialisasi basis data SQLite lokal, menjalankan migrasi, dan mengeksekusi `php artisan test`.

2. **Frontend Build:** Menyiapkan lingkungan Node.js 20, memasang pustaka menggunakan `npm ci`, dan memverifikasi kompilasi aset melalui `npm run build`.

## 3.4 Continuous Deployment ke GHCR (CPMK 3)

Soal nomor 4 mewajibkan penambahan pekerjaan penerapan (*deploy job*) pada alur kerja GitHub Actions guna mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`).

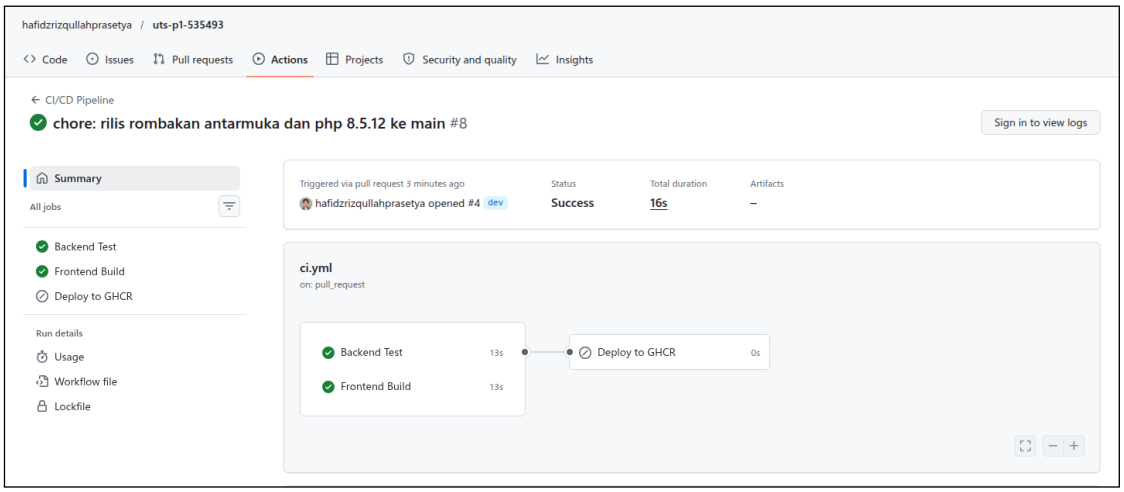
### 3.4.1 Pembatasan Eksekusi Deploy pada Cabang Utama

Pekerjaan penerapan dikonfigurasi agar hanya berjalan saat terjadi peristiwa pengunggahan (*push*) ke cabang `main`. Pada pengujian Pull Request atau cabang `dev`, pekerjaan deploy dilewati secara otomatis. Potongan logika tersebut tertulis pada Listing 5.

```
deploy:
  name: Deploy to GHCR
  needs: [backend-test, frontend-build]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
```

Listing 5: Konfigurasi bersyarat pada pekerjaan deploy di ci.yml.

Eksekusi alur kerja untuk Pull Request cabang `dev` terekam pada Gambar 9: pekerjaan pengujian backend dan kompilasi frontend selesai dengan status hijau, sementara pekerjaan deploy dilewati (*skipped*).




Gambar 9: Riwayat eksekusi GitHub Actions pada Pull Request cabang dev yang melewati tahap deploy.

### 3.4.2 Manajemen Rahasia dengan GitHub Secrets

Kunci aplikasi Laravel `APP_KEY` disimpan pada menu *Settings* → *Secrets and variables* → *Actions*, sehingga nilainya tidak pernah tertulis di repositori publik. Pekerjaan `Backend Test` membaca nilai itu sebagai variabel lingkungan; langkah `key:generate` hanya berjalan

sebagai cadangan jika secret kosong, misalnya pada fork. Login ke `ghcr.io` juga memakai secret bawaan `GITHUB_TOKEN` (Listing 6).

Kunci aplikasi sengaja tidak dimasukkan ke citra kontainer. Citra backend dipublikasikan secara publik di GHCR, sehingga kunci yang dibakar ke dalamnya, baik lewat *build-args* maupun `key:generate` saat *build*, dapat dibaca siapa saja yang mengunduh citra. Kunci diberikan saat kontainer dijalankan, misalnya `docker run -e APP_KEY=... ghcr.io/...`.

```
backend-test:
  env:
    APP_KEY: ${{ secrets.APP_KEY }}
  steps:
    - name: Generate Application Key (fallback)
      run: if [ -z "$APP_KEY" ]; then php artisan key:generate; fi

deploy:
  steps:
    - name: Log in to GitHub Container Registry (ghcr.io)
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
```

Listing 6: Pemakaian GitHub Secrets pada ci.yml.

### 3.4.3 Tag Citra Berdasarkan Commit SHA dan Registri GHCR

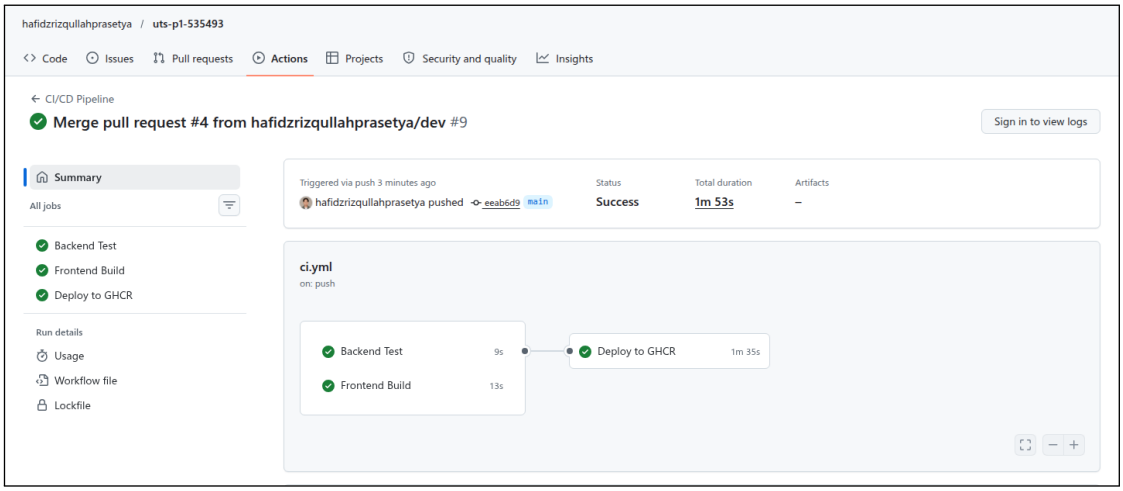
Pekerjaan penerapan melakukan otentikasi ke `ghcr.io` menggunakan token bawaan `GITHUB_TOKEN` dengan izin `packages: write`. Citra kontainer backend dan frontend dibangun dan diunggah dengan tag yang merujuk pada hash commit penuh (`${{ github.sha }}`), bukan tag `latest`. Pemberian tag unik tersebut menjamin ketertelusuran artefak rilis dengan versi kode sumber yang menyusunnya.

Gambar 10 memperlihatkan eksekusi alur kerja setelah Pull Request #4 digabung ke `main`: pengujian backend dan kompilasi frontend lulus, lalu pekerjaan deploy berjalan dan selesai dalam 1 menit 35 detik.

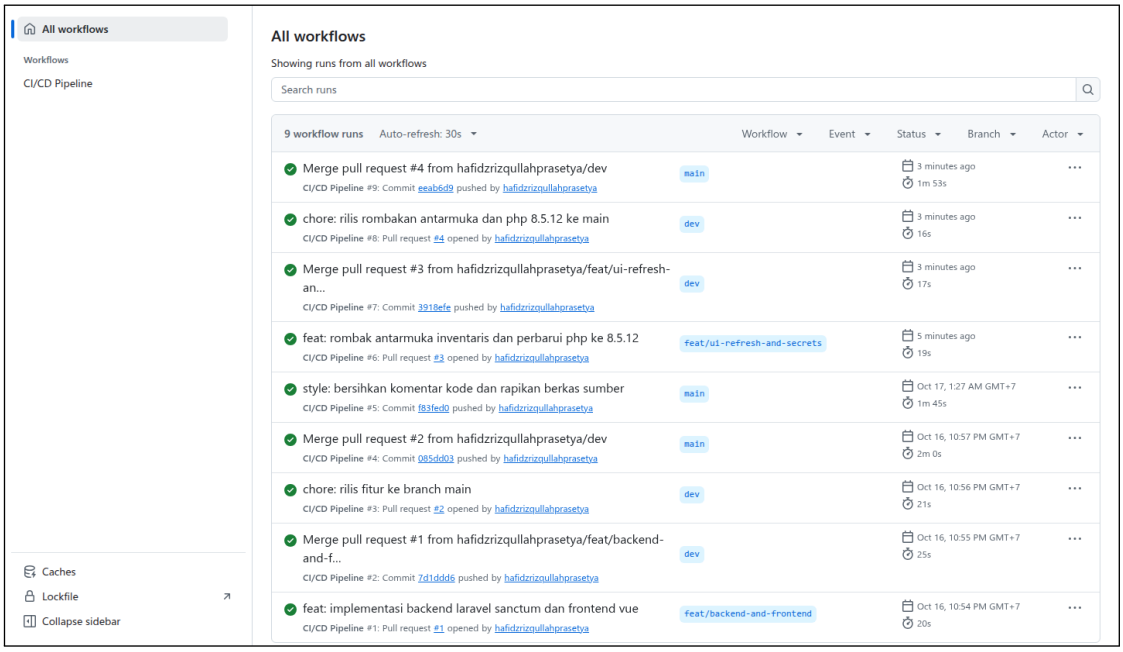
Rekapitulasi semua eksekusi, baik pengujian maupun penerapan, tersedia pada tab Actions (Gambar 11).

Kedua citra kontainer dipublikasikan pada GitHub Container Registry dengan visibilitas publik. Tab Packages repositori mendaftar tepat dua paket, yaitu `uts-p1-535493/backend` dan `uts-p1-535493/frontend` (Gambar 12).

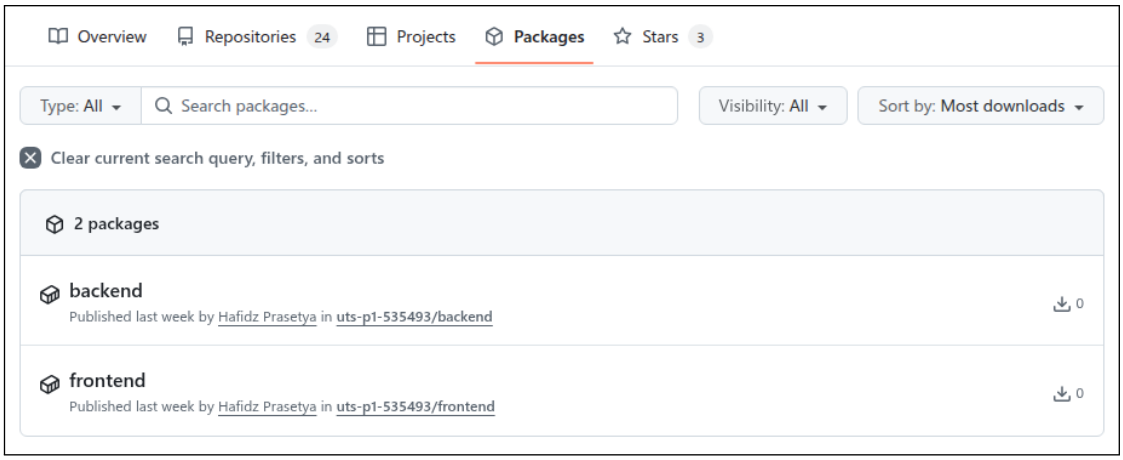
Masing-masing paket menyimpan beberapa versi yang dinamai menurut hash commit penyusunnya, bukan tag `latest`. Halaman detail paket backend dan frontend pada Gambar 13 memperlihatkan tag terbaru `eeab6d9` (komit merge Pull Request #4) di atas `f83fed0` dan




Gambar 10: Eksekusi alur kerja CI/CD pada cabang `main` dengan tahap deploy yang sukses ke GHCR.

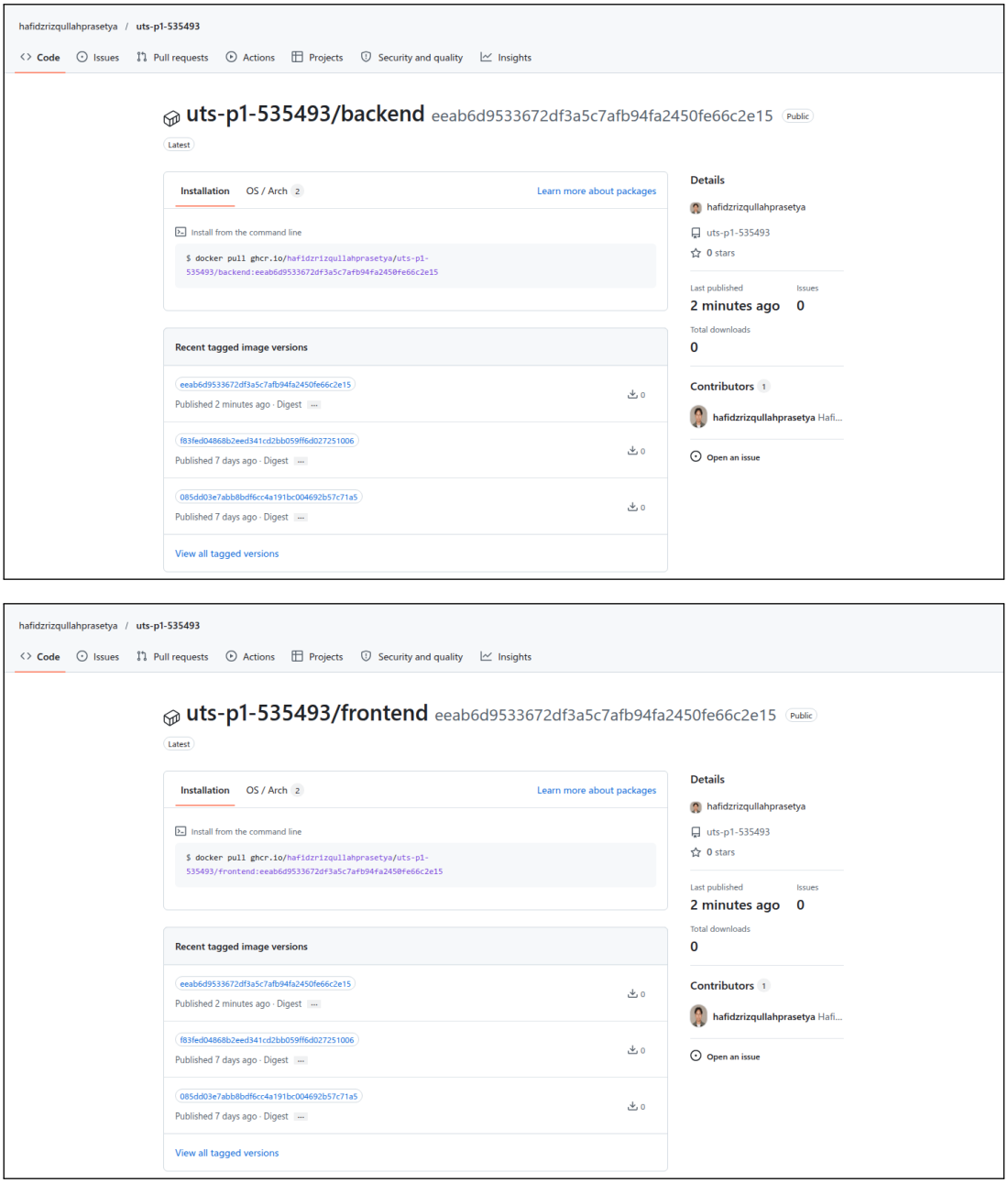



Gambar 11: Riwayat lengkap alur kerja pada tab Actions repositori GitHub.




Gambar 12: Daftar paket kontainer pada tab Packages repositori `uts-p1-535493`.

`085dd03`, beserta perintah `docker pull` yang dihasilkan GitHub, sehingga setiap citra dapat ditelusuri kembali ke komit asalnya.




Gambar 13: Halaman detail paket backend (atas) dan frontend (bawah) beserta tag commit SHA dan perintah pengunduhan citra.

## 3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5)

Soal nomor 5 meminta berkas *Dockerfile* untuk backend dan frontend yang memanfaatkan *layer caching* dan *multi-stage build*.

### 3.5.1 Dockerfile Backend dengan Optimasi Layer Caching

Berkas `backend/Dockerfile` menggunakan citra dasar resmi `php:8.5.12-cli-alpine`, yang dikunci sampai versi *patch* dan tidak memakai tag `latest`. Berkas manifes dependensi `composer.json` dan `composer.lock` disalin lebih dulu dan langsung dipasang, baru kemudian kode sumber aplikasi disalin. Dengan urutan ini, perubahan kode tidak membatalkan lapisan instalasi Composer selama manifes dependensinya tetap sama (Listing 7).

```
FROM php:8.5.12-cli-alpine

WORKDIR /var/www/html

RUN apk add --no-cache \
    curl \
    git \
    unzip \
    libzip-dev \
    sqlite-dev \
    libpng-dev \
    oniguruma-dev \
    libxml2-dev \
    linux-headers \
    && docker-php-ext-install -j$(nproc) \
    pdo \
    pdo_sqlite \
    mbstring \
    xml \
    zip \
    bcmath

COPY --from=composer:2.8 /usr/bin/composer /usr/bin/composer

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

COPY . .

RUN composer dump-autoload --optimize \
    && cp -n .env.example .env || true \
    && touch database/database.sqlite \
    && chown -R www-data:www-data storage bootstrap/cache database

EXPOSE 8000

CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

Listing 7: Berkas backend/Dockerfile dengan optimasi layer caching Composer.

### 3.5.2 Dockerfile Frontend Multi-Stage Build

Berkas `frontend/Dockerfile` mengimplementasikan arsitektur *multi-stage build*. Pada tahap pertama (`builder`), citra menggunakan `node:20-alpine` untuk memasang pustaka dependensi dan mengompilasi aset web produksi melalui `npm run build`. Pada tahap kedua, citra produksi menggunakan peladen `nginx:1.27-alpine`. Berkas hasil kompilasi dari direktori `/app/dist` disalin ke direktori publik Nginx (`/usr/share/nginx/html`), sedangkan direktori `node_modules` ditinggalkan pada tahap pembangun. Pendekatan ini menghasilkan citra kontainer akhir yang bersih dan berukuran minimal, sebagaimana dicantumkan pada Listing 8.

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:1.27-alpine

RUN rm -rf /usr/share/nginx/html/*
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Listing 8: Berkas frontend/Dockerfile menggunakan teknik multi-stage build.

# Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, dan pengujian yang telah diselesaikan pada Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak, dapat ditarik beberapa kesimpulan:

1. Backend Laravel RESTful API untuk Sistem Manajemen Aset Laboratorium melindungi seluruh rute privat dengan token Laravel Sanctum dan mengembalikan 200, 201, 422, serta 401 sesuai kondisinya. Perilaku ini dibuktikan oleh 13 pengujian otomatis (47 asersi) yang lulus dan koleksi Postman di repositori (CPMK 1).

2. Frontend Vue.js 3 dan Axios menangani login, menyimpan token di `localStorage`, dan menjalankan CRUD aset tanpa memuat ulang halaman. Alamat API dibaca dari `VITE_API_BASE_URL`, dan tampilannya tetap dapat dipakai pada layar ponsel (CPMK 1).

3. Kode dikelola di repositori publik `uts-p1-535493` dengan folder `backend/` dan `frontend/`. Cabang `main` dan `dev` mewajibkan Pull Request dan kelulusan `Backend Test` serta `Frontend Build`, dan aturan itu juga mengikat admin (CPMK 2).

4. Pekerjaan deploy di GitHub Actions hanya berjalan pada push ke `main`. `APP_KEY` dan `GITHUB_TOKEN` diambil dari GitHub Secrets, dan citra backend serta frontend dipublikasikan ke `ghcr.io` dengan tag commit SHA penuh (CPMK 3).

5. *Dockerfile* backend memakai `php:8.5.12-cli-alpine` dan memasang dependensi Composer sebelum menyalin kode agar lapisan *cache* tetap terpakai. *Dockerfile* frontend memakai *multi-stage build*, sehingga citra akhir hanya berisi Nginx dan hasil *build* tanpa `node_modules` (CPMK 4 dan CPMK 5).

# Daftar Pustaka


[1] Laravel LLC, "Laravel Sanctum - Light-weight authentication for SPAs and token APIs," 2026. [Daring]. Tersedia: https://laravel.com/framework/docs/sanctum. [Diakses: 16-Okt-2026].

[2] Vue.js Core Team, "Introduction to Vue.js - The Progressive JavaScript Framework," 2026. [Daring]. Tersedia: https://vuejs.org/guide/introduction.html. [Diakses: 16-Okt-2026].

[3] Axios Contributors, "Axios HTTP Client - Getting Started," 2026. [Daring]. Tersedia: https://axios.rest/pages/getting-started/first-steps. [Diakses: 16-Okt-2026].

[4] GitHub Inc., "About Protected Branches and Pull Request Gates," 2026. [Daring]. Tersedia: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches. [Diakses: 16-Okt-2026].

[5] GitHub Inc., "Understanding GitHub Actions - Workflow Automation and CI/CD," 2026. [Daring]. Tersedia: https://docs.github.com/en/actions/get-started/understand-github-actions. [Diakses: 16-Okt-2026].

[6] Docker Inc., "Multi-stage Builds - Optimize Container Image Sizes and Security," 2026. [Daring]. Tersedia: https://docs.docker.com/build/building/multi-stage/. [Diakses: 16-Okt-2026].

[7] Nginx Inc., "Nginx Beginner's Guide - Web Server Configuration for Static Applications," 2026. [Daring]. Tersedia: https://nginx.org/en/docs/beginners_guide.html. [Diakses: 16-Okt-2026].